

GPUMODE Lecture Study Notes: Lecture 42

Mosaic GPU

Core Problem the Lecture Addresses

This lecture explains why modern machine learning systems need programmable kernel DSLs that expose low-level accelerator features without forcing engineers to write raw assembly, PTX, or extremely verbose C++ metaprograms. The central topic is **Mosaic GPU**, a low-level GPU programming system integrated with JAX, and its relationship to **Pallas**, a Python-facing kernel DSL.

The core problem is the following:

How can we write extremely fast Hopper and Blackwell kernels from Python while retaining enough control o

The lecture argues that the role of ML languages has changed. Earlier systems such as PyTorch prioritized generality, usability, and research velocity. A 20% or 30% performance loss could be acceptable if it made model exploration easier. Today, transformer workloads dominate large-scale training and inference. These workloads are structurally simpler than arbitrary numerical programs, but they run at such large scale that a few percent of performance matters.

Large-scale cost $\approx$ devices $\times$ runtime $\times$ power $\times$ opportunity cost.

For this reason, kernel DSLs are not mainly productivity tools. They are tools engineers reach for when plain PyTorch, JAX, XLA, or vendor libraries do not give enough performance or flexibility.

The lecture presents Mosaic GPU and Pallas as an attempt to make kernels:

- **Fast enough** to approach state-of-the-art performance on Hopper-class GPUs.
- **Python-integrated** so that kernel development can happen inside the ML ecosystem.

- **Information dense** so that the program mostly contains meaningful performance decisions, not boilerplate.
- **Low-level enough** to expose TMA, WGMMMA, warp specialization, block clusters, register allocation, and hardware-specific tricks.
- **Hackable** so that researchers can inspect, modify, and extend the compiler and DSL.

The design philosophy is not to hide the hardware. The design philosophy is to hide the boring boilerplate while exposing the decisions that actually affect performance.

Visual Concept

Draw a spectrum from high-level PyTorch and JAX on the left, vendor libraries such as cuBLAS and cuDNN in the middle, and raw PTX or SASS on the far right. Place Pallas and Mosaic GPU between Triton and raw PTX, emphasizing Python usability plus lower-level hardware access.

Required Background

Why Kernel DSLs Exist

Prepackaged libraries are excellent when their abstractions match the workload exactly. For example, cuBLAS GEMM or FlashAttention kernels can be extremely optimized. The problem is that researchers and systems engineers often need slight variations:

- fused elementwise operations,
- custom attention masks,
- unusual quantization layouts,
- mixed precision accumulation,
- special memory layouts,
- custom scheduling around communication or memory transfer,
- experimental tensor core usage.

If the library does not support the desired variant, the engineer can fall off a steep performance cliff.

A general compiler can sometimes help, but highly optimizing compilers are hard to adapt quickly to new hardware. The lecture emphasizes that hardware evolved significantly from Ampere to Hopper to Blackwell. Each generation changes the optimal programming model.

Compiler usefulness = generality + optimization quality + hardware freshness.

The problem is that the third term changes quickly. A compiler that is excellent for Ampere may not automatically produce optimal Hopper kernels, especially when using new features such as TMA and WGMMMA.

Python as a Kernel Front End

The lecture argues that Python can be a reasonable front end for generating fast kernels. The input language does not determine peak performance. What matters is the generated IR, PTX, and final SASS.

Fast kernel $\neq$ kernel written directly in C++.

Instead:

Fast kernel $\equiv$ good generated code+good memory schedule+good hardware utilization.

Python is attractive because metaprogramming is simpler than C++ templates or string-based code generation. Kernel development often involves compile-time decisions such as tile sizes, unrolling, layout transforms, pipeline depth, and specialization based on dtype or shape.

Tracing Versus AST Parsing

The lecture contrasts several programming models:

1. **C++ template metaprogramming:** powerful but difficult to debug and often produces poor error messages.
2. **String-based code generation:** flexible but fragile because variable scope, name capture, and code hygiene become difficult.
3. **AST-based Python DSLs:** parse Python syntax and stage it into another IR. Triton follows this broad style.
4. **Tracing-based DSLs:** run the Python function once with traced values to build a representation of the computation. JAX and Pallas follow this approach.

In tracing-based systems, native Python control flow is evaluated at tracing time. This means a Python loop may become unrolled code.

For ordinary JAX programs, this can be annoying because runtime control flow must use special combinators. For kernels, this can be useful:

Python for loop during tracing $\rightarrow$ compile-time unrolled kernel body.

This makes Python itself the metaprogramming language.

For kernel programming, tracing turns ordinary Python control flow into a natural compile-time code-generation mechanism.

Modern GPU Execution Concepts

The lecture assumes familiarity with these GPU concepts:

- **SM**: streaming multiprocessor, the unit that schedules and executes thread blocks.
- **Warp**: a group of CUDA threads executing in SIMD-like fashion.
- **Warp group**: on Hopper, a larger cooperative unit often used for WG-MMA.
- **Thread block**: CUDA execution unit scheduled onto one SM.
- **Grid**: collection of independent blocks.
- **Global memory**: high-capacity HBM, high latency and high bandwidth.
- **Shared memory**: programmer-managed on-chip memory within an SM.
- **Registers**: fastest per-thread storage.
- **Tensor cores**: matrix multiply hardware units.
- **ALU**: scalar or vector arithmetic units, used for operations such as softmax.
- **Occupancy**: amount of active work resident on an SM.
- **Pipelining**: overlapping memory transfer and compute.

Main Concepts

Mosaic GPU and Pallas

Mosaic GPU is an open-source GPU programming project inside the JAX repository. It is designed for writing low-level kernels for modern NVIDIA GPUs. Pallas is a higher-level Python-facing DSL that can target multiple backends.

The lecture describes Pallas as having multiple backends:

- Triton backend,
- Mosaic TPU backend,
- Mosaic GPU backend.

Mosaic GPU began as a direct low-level DSL built on MLIR Python builder APIs. Over time, the team decided to push users more toward Pallas because it provides a better user experience while still connecting to Mosaic GPU for low-level code generation.

Pallas $\longrightarrow$ Mosaic GPU backend $\longrightarrow$ GPU-specific low-level code.

What Is a Mosaic GPU Thread?

The word “thread” is overloaded. In CUDA, a thread usually means a scalar lane inside a warp. In Mosaic GPU, the lecture uses **thread** to mean something closer to a **warp group**.

This is a deliberate abstraction choice.

For dense linear algebra, exposing individual CUDA lanes is often too low-level. Exposing an entire block, as Triton often does, can sometimes be too high-level. Mosaic GPU chooses a warp-group-level abstraction because modern matrix instructions such as WGMMMA operate at that granularity.

Mosaic thread $\approx$ CUDA warp group.

The reasoning is:

- Tensor core instructions are increasingly large.
- Dense linear algebra usually needs cooperative groups of lanes.
- A warp-group abstraction exposes enough hardware detail without forcing lane-level programming.

Pallas References

In ordinary JAX, arrays are immutable values. In Pallas kernels, arguments are **references**. A reference is shaped, mutable, and points to a memory region.

A typical Pallas kernel has the form:

```
def add_kernel(x_ref, y_ref, z_ref):  
    z_ref[...] = x_ref[...] + y_ref[...]
```

The expression `x_ref[...]` means: load the current value from the reference. It is similar to dereferencing a pointer, but with shape information.

`x_ref[...]` means load shaped value from reference.

The result of reading from a reference is an immutable array value. Inside a kernel, this usually corresponds conceptually to data living in registers.

$$x_{\text{reg}} \leftarrow \text{load}(x_{\text{ref}}).$$

Then compute happens on register values:

$$z_{\text{reg}} = x_{\text{reg}} + y_{\text{reg}}.$$

Finally, the result is stored back:

$$\text{store}(z_{\text{ref}}, z_{\text{reg}}).$$

Calling Pallas Kernels from JAX

JAX normally works with arrays, not references. A Pallas kernel must be wrapped so that it becomes callable from JAX.

A simplified pattern is:

```
import jax
import jax.numpy as jnp
from jax.experimental import pallas as pl

def add_kernel(x_ref, y_ref, z_ref):
    z_ref[...] = x_ref[...] + y_ref[...]

jax_add = pl.pallas_call(
    add_kernel,
    out_shape=jax.ShapeDtypeStruct(x_shape, x_dtype),
)
```

The output shape must be declared because Pallas allocates the output buffer.

$$\text{kernel arguments} = \text{input references} + \text{output references}.$$

If the Python wrapper takes two input arrays, the kernel body may receive three references: two inputs and one output.

Grids and Program IDs

To exploit parallelism, the same kernel body is invoked over a grid. A grid is a tuple of integers. The product of the tuple entries gives the number of independent program instances.

$$N_{\text{programs}} = \prod_{i=0}^{d-1} G_i,$$

where G_i is the grid size along axis i .

For a one-dimensional grid:

$$\text{pid} = \text{program_id}(0).$$

Each program instance can process a different slice of data:

$$\text{start} = \text{pid} \cdot B, \quad \text{end} = \text{start} + B,$$

where B is the block size.

Representative code:

```
def add_kernel(x_ref, y_ref, z_ref):
    pid = pl.program_id(0)
    block = pl.ds(pid * block_size, block_size)
    z_ref[block] = x_ref[block] + y_ref[block]
```

Pallas uses shaped slicing rather than pointer arithmetic. This reduces indexing noise and gives the compiler more structure.

Block Specifications

For more advanced kernels, Pallas can predeclare how each program instance accesses inputs and outputs. This is done with **block specifications**.

A block specification has two conceptual parts:

1. the shape of the tile,
2. a mapping from grid indices to tile coordinates.

For matrix multiplication $C = AB$, suppose:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

A simple tiled output block has shape:

$$C_{\text{tile}} \in \mathbb{R}^{B_M \times B_N}.$$

The grid can be:

$$G_M = \left\lceil \frac{M}{B_M} \right\rceil, \quad G_N = \left\lceil \frac{N}{B_N} \right\rceil.$$

For program coordinates (i, j) , the output tile is:

$$C_{i,j} = C[iB_M : (i+1)B_M, jB_N : (j+1)B_N].$$

The corresponding input tiles may be:

$$A_i = A[iB_M : (i+1)B_M, 0 : K],$$

$$B_j = B[0 : K, jB_N : (j+1)B_N].$$

The lecture emphasizes that this access pattern is not inferred by a heroic compiler optimization. It is explicitly declared by the programmer. This declaration allows the backend to implement predictable pipelining.

Declared access pattern $\implies$ known future memory transfers $\implies$ automatic pipelining.

Pre-Sliced References

When block specifications are used, the kernel body receives references that already correspond to the selected tile, not the full input array. This means the kernel body can be much cleaner.

Instead of writing complicated indexing inside the kernel, the indexing is factored into the block specification.

The kernel body says what computation happens on a tile. The block specification says which tile each program instance receives.

This separation is important because transformer kernels often have structured access patterns. The compiler can use the declared structure to manage asynchronous copies and pipeline stages.

Hardware-Level Explanation

Why Modern GPUs Look More TPU-Like

The lecture argues that modern datacenter GPUs and TPUs are converging architecturally. TPUs began as matrix multiplication accelerators. GPUs began as graphics accelerators and then became general-purpose parallel processors. Modern machine learning workloads pushed GPUs toward larger matrix engines, asynchronous memory movement, and more explicit scheduling.

The convergence includes:

- larger tensor core instructions,
- bigger cooperative execution units,

- more explicit memory movement,
- asynchronous HBM-to-shared-memory transfers,
- lower tolerance for relying only on occupancy to hide latency,
- more explicit pipeline construction,
- separate scalar or uniform compute resources.

Historically, GPUs hid memory latency using many resident warps. If one warp stalled on global memory, another warp could run.

Latency hiding $\approx$ many independent resident warps.

However, modern tensor core kernels often consume so many registers and shared memory bytes that only a small number of blocks can fit on an SM. Sometimes only one block fits.

large tile + many registers + large shared memory $\implies$ low occupancy.

When occupancy is low, latency hiding by massive oversubscription is less effective. The kernel must explicitly overlap memory movement and compute.

Modern performance $\approx$ explicit pipeline+tensor core saturation+controlled memory movement.

Global Memory, Shared Memory, and Registers

Mosaic GPU exposes memory spaces rather than hiding them.

The relevant hierarchy is:

HBM/global memory $\rightarrow$ L2 $\rightarrow$ shared memory $\rightarrow$ registers $\rightarrow$ compute units.

Approximate roles:

- **Global memory:** stores large tensors, high latency, high bandwidth.
- **Shared memory:** explicitly managed on-chip memory used for tiles.
- **Registers:** fastest storage, used for accumulators and temporary values.
- **Tensor cores:** matrix multiply units, often consuming operands from shared memory.
- **ALUs:** scalar and vector compute units for operations such as softmax.

A fast kernel manages these spaces deliberately. FlashAttention is fast largely because it avoids materializing the full attention matrix in HBM and instead keeps partial results inside on-chip memory.

Fast attention $\approx$ avoid HBM traffic for intermediate QK^T and softmax.

TMA: Tensor Memory Accelerator

Hopper introduced TMA, an asynchronous copy mechanism that can move multidimensional tiles between global memory and shared memory. Mosaic GPU exposes this through functions such as conceptual global-to-shared copies.

A simplified pattern is:

```
def scoped(smem_lhs, smem_rhs, smem_out, barrier):
    plgpu.copy_gmem_to_smem(lhs_ref.at[lhs_slice], smem_lhs, barrier)
    plgpu.copy_gmem_to_smem(rhs_ref.at[rhs_slice], smem_rhs, barrier)
    plgpu.barrier_wait(barrier)
    smem_out[...] = compute(smem_lhs[...], smem_rhs[...])
    plgpu.copy_smem_to_gmem(smem_out, out_ref.at[out_slice])
```

The actual APIs may differ slightly, but the concept is:

HBM tile $\xrightarrow{\text{TMA}}$ shared memory tile $\xrightarrow{\text{compute}}$ shared memory output $\xrightarrow{\text{copy}}$ HBM output.

TMA is powerful but not user-friendly at the raw level. It requires descriptors, synchronization, barriers, and careful memory layout. Mosaic GPU tries to automate the boilerplate while preserving the performance-relevant choices.

Barriers and Asynchronous Copies

Asynchronous copies require synchronization. If a warp group starts computing before the copy finishes, it may read invalid shared memory. If a memory warp overwrites shared memory before compute warp groups finish reading it, correctness breaks.

The synchronization pattern is:

producer arrives $\rightarrow$ consumer waits $\rightarrow$ consumer computes $\rightarrow$ consumer arrives $\rightarrow$ producer may overwrite.

In GPU terms:

- a memory warp group issues copies,
- compute warp groups wait for the data,

- compute warp groups signal when done,
- memory warp group reuses shared memory buffers.

WGMMMA: Warp Group Matrix Multiply Accumulate

WGMMMA is Hopper’s warp-group-level matrix multiply instruction family. It allows a group of warps to cooperatively execute tensor core matrix multiplication.

Conceptually:

$$D \leftarrow AB + D,$$

where:

$$A \in \mathbb{R}^{M_{\text{tile}} \times K_{\text{tile}}}, \quad B \in \mathbb{R}^{K_{\text{tile}} \times N_{\text{tile}}}, \quad D \in \mathbb{R}^{M_{\text{tile}} \times N_{\text{tile}}}.$$

In Mosaic GPU, operands are typically in shared memory and the accumulator may be represented as a reference-like object backed by registers or a compiler-managed fragment.

Representative structure:

```
def compute(smem_lhs, smem_rhs, smem_out):
    def body(acc_ref):
        plgpu.wgmma(acc_ref, smem_lhs, smem_rhs)
        smem_out[...] = acc_ref[...]

    pl.run_scoped(
        body,
        plgpu.ACC((bm, bn), jnp.float32),
    )
```

The important idea is not the exact syntax but the memory and hardware behavior:

- operands are staged in shared memory,
- WGMMMA reads shared-memory tiles,
- tensor cores perform matrix multiply,
- accumulation happens in a register-like accumulator,
- output is written back to shared memory or global memory.

Swizzles and Tiling Transforms

Shared memory has banks. If many lanes access the same bank, accesses serialize and performance drops.

bank conflict $\implies$ shared memory serialization $\implies$ lower throughput.

Mosaic GPU allows transformations such as swizzles and tiling transforms on block specifications. These change how data is laid out in shared memory.

Suppose a matrix tile is logically:

$$128 \times 128.$$

Instead of storing it as ordinary row-major order, the data may be tiled into subtiles such as:

$$64 \times 32.$$

The memory order becomes:

row-major within each 64×32 tile, then row-major across tiles.

This can match tensor core operand expectations and reduce bank conflicts. The lecture highlights that TMA can sometimes perform such reshuffling during the copy from global memory to shared memory.

TMA copy+layout transform $\implies$ data arrives in tensor-core-friendly shared-memory layout.

Block Clusters and TMA Multicast

Hopper also supports block clusters and TMA multicast. A cluster is a group of blocks that can cooperate more closely than ordinary independent CUDA blocks. TMA multicast can copy one global memory tile into the shared memory of multiple blocks.

Conceptually:

$$\text{one global tile} \rightarrow \begin{cases} \text{shared memory of block 0,} \\ \text{shared memory of block 1.} \end{cases}$$

This is surprising because traditional CUDA programming treats blocks as independent. Multicast allows multiple blocks to receive the same data efficiently.

The lecture's key distinction is:

Deciding whether to use multicast = hard performance decision,

but:

emitting the required low-level instructions = mostly mechanical boilerplate.

Mosaic GPU tries to automate the second while exposing the first.

Warp Specialization

Warp specialization assigns different warp groups within a block to different roles. A common pattern is:

- one warp group handles memory transfer,
- two or more warp groups handle compute.

For FlashAttention 3 style kernels, one warp group may load K and V tiles while other warp groups compute attention.

Without a helper, the kernel may contain explicit conditionals:

```
wg = lax.axis_index("workgroups")

if wg < 2:
    # Compute warp groups.
    # Wait for data, run QK, softmax, PV.
    pass
else:
    # Memory warp group.
    # Issue async copies into shared memory.
    pass
```

This is powerful but error-prone. The load happens far from the use of the data. Barriers must be manually paired. Register allocation must be adjusted.

Mosaic GPU and Pallas are adding helpers such as warp-specialized pipelines so that the programmer declares the access pattern and compute body, while the backend generates the memory warp group logic.

warp-specialized pipeline helper = access pattern+compute function+generated memory producer.

Mathematical Reasoning

Arithmetic Intensity

A recurring performance model is arithmetic intensity:

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

A kernel is likely memory-bound when:

$$I < \frac{\text{peak FLOPs}}{\text{peak memory bandwidth}}.$$

A kernel is likely compute-bound when:

$$I > \frac{\text{peak FLOPs}}{\text{peak memory bandwidth}}.$$

For matrix multiplication:

$$C = AB,$$

where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}.$$

The FLOP count is approximately:

$$\text{FLOPs} = 2MKN.$$

The minimum data movement for reading A , reading B , and writing C is:

$$\text{Bytes} = s(MK + KN + MN),$$

where s is bytes per element. Thus:

$$I_{\text{matmul}} = \frac{2MKN}{s(MK + KN + MN)}.$$

Large GEMMs have high arithmetic intensity, making tensor core utilization the critical bottleneck.

Tiled Matrix Multiplication

For tiled matrix multiplication, each output tile is:

$$C_{i,j} = \sum_{t=0}^{T-1} A_{i,t} B_{t,j}.$$

The tile shapes may be:

$$A_{i,t} \in \mathbb{R}^{B_M \times B_K}, \quad B_{t,j} \in \mathbb{R}^{B_K \times B_N}, \quad C_{i,j} \in \mathbb{R}^{B_M \times B_N}.$$

The number of K -tiles is:

$$T = \left\lceil \frac{K}{B_K} \right\rceil.$$

The accumulator update is:

$$C_{i,j}^{(t+1)} = C_{i,j}^{(t)} + A_{i,t} B_{t,j}.$$

This maps naturally to a pipeline:

$$\text{load } A_{i,t+1}, B_{t+1,j} \quad \text{while computing} \quad A_{i,t} B_{t,j}.$$

Pipeline Overlap Model

Let:

$$L_t = \text{time to load tile } t,$$

$$C_t = \text{time to compute tile } t,$$

$$S_t = \text{time to store output tile } t.$$

Without overlap:

$$T_{\text{serial}} = \sum_t (L_t + C_t + S_t).$$

With ideal overlap between load and compute:

$$T_{\text{pipeline}} \approx \sum_t \max(L_t, C_t) + \text{pipeline fill and drain overhead.}$$

The goal is to hide L_t behind C_t :

$$C_t \geq L_t \implies \text{memory transfer mostly hidden.}$$

If:

$$L_t > C_t,$$

then compute units wait for memory and the kernel is memory-bound or pipeline-limited.

FlashAttention Online Softmax

For attention:

$$S = QK^T, \quad P = \text{softmax}(S), \quad O = PV.$$

Naive attention materializes S and P , which requires large HBM traffic:

$$S \in \mathbb{R}^{L \times L}.$$

FlashAttention avoids writing the full S and P matrices to HBM. It computes attention block by block while maintaining online softmax statistics.

For a row of scores s_j , define:

$$m = \max_j s_j,$$

$$\ell = \sum_j \exp(s_j - m),$$

$$o = \sum_j \exp(s_j - m) v_j.$$

When processing a new tile with local maximum m_{new} , combine it with old state:

$$m' = \max(m, m_{\text{new}}).$$

The old normalizer rescales as:

$$\ell'_{\text{old}} = \ell \exp(m - m').$$

The new tile normalizer is:

$$\ell'_{\text{tile}} = \sum_j \exp(s_j - m').$$

Then:

$$\ell' = \ell'_{\text{old}} + \ell'_{\text{tile}}.$$

The output accumulator rescales similarly:

$$o' = o \exp(m - m') + \sum_j \exp(s_j - m') v_j.$$

The final output is:

$$O = \frac{o'}{\ell'}.$$

This algorithm reduces memory traffic by keeping intermediate statistics in registers and shared memory.

Head Dimension and Tensor Core Utilization

The profiler demo compares head dimension 64 and 256. The key point is that increasing the head dimension increases the cost of the matrix multiplications, but the softmax cost does not grow in the same way.

For attention with sequence tile size B and head dimension D , the approximate QK work is:

$$\text{FLOPs}_{QK} \approx 2B^2D.$$

The PV work is similarly:

$$\text{FLOPs}_{PV} \approx 2B^2D.$$

Softmax over a $B \times B$ score tile costs approximately:

$$\text{Ops}_{\text{softmax}} = O(B^2),$$

not $O(B^2D)$.

Thus:

$$\frac{\text{matmul work}}{\text{softmax work}} \propto D.$$

When $D = 64$, softmax may be relatively expensive, causing tensor cores to wait. When $D = 256$, matmul dominates, so tensor cores can remain busy more consistently.

Code Walkthrough

Minimal Pallas Add Kernel

```
import jax
import jax.numpy as jnp
from jax.experimental import pallas as pl

def add_kernel(x_ref, y_ref, z_ref):
    z_ref[...] = x_ref[...] + y_ref[...]

x = jnp.ones((1024,), dtype=jnp.float32)
y = jnp.ones((1024,), dtype=jnp.float32)

add = pl.pallas_call(
    add_kernel,
    out_shape=jax.ShapeDtypeStruct(x.shape, x.dtype),
)

z = add(x, y)
```

Explanation

- `x_ref`, `y_ref`, and `z_ref` are shaped references.
- `x_ref[...]` loads the whole referenced region.
- The addition happens on loaded values, conceptually in registers.
- `z_ref[...]` stores the result.
- `out_shape` tells Pallas what output buffer to allocate.

This kernel has little parallelism unless wrapped with a grid. It is useful mainly for understanding the reference model.

Grid-Based Add Kernel

```
def add_kernel(x_ref, y_ref, z_ref):
    pid = pl.program_id(0)
    block = pl.ds(pid * block_size, block_size)
    z_ref[block] = x_ref[block] + y_ref[block]

grid = (num_blocks,)

add = pl.pallas_call(
    add_kernel,
    out_shape=jax.ShapeDtypeStruct(x.shape, x.dtype),
    grid=grid,
)
```

Explanation

Each program instance receives a different program ID. The program ID determines which slice of the vector it processes.

$$\text{slice}_p = [pB, pB + B).$$

For contiguous arrays, adjacent program instances access adjacent memory ranges. This is good for global memory coalescing at the block or warp-group level, depending on the backend mapping.

Matmul with Block Specifications

```
def matmul_kernel(x_ref, y_ref, z_ref):
    z_ref[...] = jnp.dot(x_ref[...], y_ref[...])

matmul = pl.pallas_call(
    matmul_kernel,
    out_shape=jax.ShapeDtypeStruct((m, n), jnp.float32),
    grid=(m // bm, n // bn),
    in_specs=[
        pl.BlockSpec((bm, k), lambda i, j: (i, 0)),
        pl.BlockSpec((k, bn), lambda i, j: (0, j)),
    ],
    out_specs=pl.BlockSpec((bm, bn), lambda i, j: (i, j)),
)
```

Explanation

The kernel body is short because the references are already sliced.

For program coordinate (i, j) :

$$x_{\text{ref}} \equiv A[iB_M : (i+1)B_M, 0 : K],$$

$$y_{\text{ref}} \equiv B[0 : K, jB_N : (j+1)B_N],$$

$$z_{\text{ref}} \equiv C[iB_M : (i+1)B_M, jB_N : (j+1)B_N].$$

This example assumes the K dimension is small enough to handle in one tile. A full high-performance matmul would also tile over K .

Explicit Shared Memory Allocation

```
def kernel(lhs_ref, rhs_ref, out_ref):
    def scoped(lhs_smem, rhs_smem, out_smem, barrier):
        plgpu.copy_gmem_to_smem(lhs_ref.at[lhs_slice], lhs_smem, barrier)
        plgpu.copy_gmem_to_smem(rhs_ref.at[rhs_slice], rhs_smem, barrier)

        plgpu.barrier_wait(barrier)

        out_smem[...] = lhs_smem[...] + rhs_smem[...]

        plgpu.copy_smem_to_gmem(out_smem, out_ref.at[out_slice])

    pl.run_scoped(
        scoped,
        plgpu.SMEM(lhs_shape, lhs_dtype),
        plgpu.SMEM(rhs_shape, rhs_dtype),
        plgpu.SMEM(out_shape, out_dtype),
        plgpu.Barrier(),
    )
```

Explanation

- `pl.run_scoped` creates scoped allocations.
- `SMEM` allocations live in shared memory.
- `copy_gmem_to_smem` uses an asynchronous copy path.
- A barrier ensures the data has arrived before compute starts.
- The result is written to shared memory and then copied back to global memory.

This example is intentionally simple. It schedules async copies and immediately waits, so it does not get much overlap. It shows the explicit memory model.

Using a Pipeline Helper

```
def compute(lhs_smem, rhs_smem, out_smem):
    out_smem[...] = lhs_smem[...] + rhs_smem[...]

plgpu.emit_pipeline(
    compute,
    in_specs=[
        pl.BlockSpec((rows_per_block, cols_per_step),
                      lambda block_i, step: (block_i, step)),
        pl.BlockSpec((rows_per_block, cols_per_step),
                      lambda block_i, step: (block_i, step)),
    ],
    out_specs=pl.BlockSpec((rows_per_block, cols_per_step),
                           lambda block_i, step: (block_i, step)),
    grid=(num_blocks,),
    num_steps=num_steps,
)
```

Explanation

The helper automates the loop that overlaps memory movement and compute. The programmer declares:

- how many pipeline steps exist,
- which input slices are needed at each step,
- which output slice is produced,
- what computation happens on the shared-memory tiles.

The backend handles:

- shared-memory staging,
- async copy scheduling,
- barriers,
- double buffering or equivalent buffering strategy,
- ordering of copy and compute.

The performance model becomes:

$$T \approx T_{\text{fill}} + \sum_t \max(L_t, C_t) + T_{\text{drain}}.$$

WGMMA Compute Skeleton

```
def compute(lhs_smem, rhs_smem, out_smem):
    def scoped(acc):
        acc[...] = jnp.zeros(acc.shape, acc.dtype)
        plgpu.wgmma(acc, lhs_smem, rhs_smem)
        out_smem[...] = acc[...]

    pl.run_scoped(
        scoped,
        plgpu.ACC((bm, bn), jnp.float32),
    )
```

Explanation

- `lhs_smem` and `rhs_smem` are tensor core operands staged in shared memory.
- `acc` is an accumulator fragment.
- `wgmma` invokes the warp-group-level matrix multiply accumulate instruction.
- The output accumulator is eventually stored.

For a tiled loop over K , this would run repeatedly:

$$\text{acc} \leftarrow \text{acc} + A_{i,t} B_{t,j}.$$

Warp-Specialized Attention Structure

```
def flash_attention_kernel(q_ref, k_ref, v_ref, o_ref):
    wg = lax.axis_index("workgroups")

    if wg < num_compute_wgs:
        # Compute warp groups:
        # 1. wait for K and V tiles
        # 2. compute QK
        # 3. compute online softmax
        # 4. compute P times V
        # 5. signal that shared memory can be reused
        pass
    else:
        # Memory warp group:
        # 1. issue async copies for K and V
        # 2. wait until compute warp groups release buffers
        # 3. advance pipeline
        pass
```

Explanation

This explicit style works, but it separates the memory load from the place where the data is used. That makes it hard to maintain.

The improved helper-based style is conceptually:

```
def compute(k_smem, v_smem, done_barrier):
    scores = q_tile @ k_smem.T
    probs = online_softmax(scores)
    out_acc = probs @ v_smem
    plgpu.barrier_arrive(done_barrier)

plgpu.emit_pipeline_warp_specialized(
    compute,
    in_specs=[k_block_spec, v_block_spec],
    num_compute_wgs=2,
    num_memory_wgs=1,
)
```

The point is that the compute function directly receives the shared-memory operands it needs. The producer warp group is generated from the declared block specs.

Performance Intuition

Why Pipelining Is Central

On modern GPUs, performance is often determined by whether the kernel keeps all important hardware resources busy at the same time.

Relevant resources include:

- HBM bandwidth,
- TMA copy engines,
- shared memory bandwidth,
- tensor cores,
- ALUs,
- register file,
- interconnect for multi-GPU kernels.

A naive kernel may run these resources sequentially:

load $\rightarrow$ compute $\rightarrow$ store.

A high-performance kernel overlaps them:

load tile 0	load tile 1	load tile 2	...
	compute tile 0	compute tile 1	...
		store tile 0	...

The kernel is fast when the longest stage dominates and the other stages are hidden.

$$T_{\text{steady}} \approx \max(T_{\text{load}}, T_{\text{compute}}, T_{\text{store}}).$$

Why FlashAttention 3 Uses Warp Specialization

FlashAttention has both matrix multiplication and softmax. Matrix multiplication uses tensor cores. Softmax uses ALUs.

If both compute warp groups do matmul at the same time, they may fight for tensor cores. Then both may move to softmax and fight for ALUs. This creates resource phase imbalance.

A better schedule staggers the work:

time	warp group 0	warp group 1
t_0	QK or PV on tensor cores	softmax on ALU
t_1	softmax on ALU	QK or PV on tensor cores

This schedule attempts to use tensor cores and ALUs concurrently.

good schedule $\implies$ tensor cores busy + ALUs useful + less phase contention.

Profiler Interpretation

The lecture demonstrates a work-group-level profiler. It records events from inside the kernel into shared memory, copies them to global memory at the end, and converts them into a Chrome trace or Perfetto trace.

The trace interpretation is:

- each process corresponds to one SM,
- each line corresponds to a warp group,
- colored regions correspond to named kernel regions,
- gaps or waits reveal synchronization or memory stalls.

For FlashAttention 3, traces show repeated phases:

$$QK \rightarrow \text{softmax} \rightarrow PV.$$

The profiler can reveal whether the kernel is waiting on:

- memory arrival,
- softmax barrier synchronization,
- tensor core availability,
- ALU work,
- shared memory buffer reuse.

This is valuable because it shows actual execution behavior rather than only aggregate counters.

Head Dimension Example

For head dimension 64, softmax can be relatively expensive compared with the matmul. The profiler shows tensor cores waiting because softmax is still running.

For head dimension 256, matmul becomes more expensive, and softmax is often hidden. The profiler shows tensor core utilization improving.

This is consistent with:

$$\text{matmul work} \propto D, \quad \text{softmax work} \not\propto D.$$

Thus increasing D shifts the bottleneck toward tensor core work, which is favorable if tensor cores can be kept continuously occupied.

Why Boilerplate Should Be Automated

The lecture repeatedly distinguishes between two kinds of complexity:

1. **Essential performance decisions:** tile sizes, memory layout, pipeline strategy, warp specialization, tensor core usage, register allocation.
2. **Mechanical boilerplate:** descriptors, barrier plumbing, copy instruction details, repetitive pipeline loops, multicast instruction choreography.

The Mosaic GPU philosophy is:

automate mechanical complexity but expose essential performance choices.

This is why the language is lower-level than Triton in some places but still tries to provide helpers for common patterns.

Common Misconceptions

Misconception: Kernel DSLs Are Mainly for Convenience

Kernel DSLs are not primarily convenience tools. They are used when high-level frameworks are not fast or flexible enough.

Correct view:

start with PyTorch or JAX → profile → identify bottleneck → write custom kernel only if needed.

Misconception: Python Cannot Generate Peak-Performance Kernels

Python being the front end does not imply slow generated code. The generated IR and final machine code determine runtime performance.

Correct view:

Python DSL → MLIR or backend IR → PTX → SASS → hardware execution.

Misconception: More Automation Is Always Better

Some decisions should not be hidden. Tile size, pipeline structure, and memory layout can drastically affect performance. A compiler may not know the best strategy, especially for new hardware.

Correct view:

automate only when the automation is reliably good.

Misconception: Portability Is the Main Goal

For Mosaic GPU, portability is secondary. The goal is state-of-the-art performance on specific modern accelerators. Hardware-specific APIs are acceptable when they expose important capabilities.

peak performance > single portable abstraction.

Misconception: High Occupancy Always Means High Performance

For tensor core kernels, low occupancy can be acceptable or unavoidable. Large tiles, WGMMMA, and shared memory usage may reduce the number of resident

blocks. Performance then depends on careful pipelining rather than simply maximizing occupancy.

low occupancy $\nrightarrow$ bad performance.

Misconception: Softmax Is Always Negligible in Attention

Softmax can matter, especially for smaller head dimensions. If the tensor core work is small, ALU softmax work and synchronization can limit throughput.

D small $\implies$ softmax relatively important.

D large $\implies$ matmul dominates.

Misconception: LLMs Remove the Need for DSLs

The lecture argues that even if LLMs can generate low-level code, large-scale kernels are high-stakes. A million lines of generated PTX would be difficult to verify. A concise DSL kernel can be reviewed, understood, and modified by humans.

human-verifiable kernel matters for correctness and cost.

Practical Takeaways

Engineering Takeaways

- Use high-level frameworks first, then write kernels only for measured bottlenecks.
- For modern Hopper kernels, explicit memory movement and pipelining are central.
- Pallas separates tile computation from tile access patterns.
- Mosaic GPU exposes hardware-specific features such as TMA, WGMMMA, block clusters, multicast, register allocation, and profiling.
- Shared memory layout matters because tensor cores and bank conflicts are layout-sensitive.
- Warp specialization is a scheduling strategy for overlapping memory, tensor core work, and ALU work.
- Profiling at warp-group granularity can reveal bottlenecks that aggregate metrics hide.

- Transformer kernels are structured enough that concise high-performance DSL kernels are realistic.

Mental Checklist for a Mosaic GPU Kernel

When designing a kernel, ask:

1. What tensor slices does each program instance own?
2. Which dimensions define the grid?
3. Which operands should be loaded by TMA?
4. What lives in shared memory?
5. What lives in registers?
6. Is the computation tensor-core-heavy or ALU-heavy?
7. Can memory transfer overlap compute?
8. Is warp specialization useful?
9. Are shared memory layouts tensor-core-friendly?
10. Are barriers paired correctly?
11. Does the profiler show idle tensor cores, memory waits, or synchronization stalls?

Project or Experiment Ideas

Naive Versus Pipelined Add or Copy Kernel

Implement a simple vector or matrix operation in Pallas. Compare:

- direct global memory loads,
- explicit shared memory staging,
- pipeline helper staging.

Measure whether asynchronous copies help or whether the operation is too simple to benefit.

Tiled Matmul with WGMMA

Implement a small tiled matrix multiplication using Mosaic GPU concepts:

$$C_{i,j} = \sum_t A_{i,t} B_{t,j}.$$

Vary:

- B_M ,
- B_N ,
- B_K ,
- dtype,
- accumulator dtype,
- pipeline depth.

Profile tensor core utilization.

Shared Memory Layout Experiment

Take the same logical matrix tile and compare:

- row-major shared memory layout,
- tiled shared memory layout,
- swizzled layout.

Measure bank conflicts and tensor core throughput if possible.

FlashAttention Profiling Experiment

Implement or inspect a FlashAttention-style kernel. Annotate phases:

$$QK, \quad \text{softmax}, \quad PV.$$

Use a trace viewer to compare head dimensions:

$$D \in \{64, 128, 256\}.$$

Check whether softmax or matmul is the limiting phase.

Warp Specialization Experiment

Write two versions of a tiled kernel:

1. a non-warp-specialized version where all warp groups do similar work,
2. a warp-specialized version with memory and compute warp groups.

Compare:

- code complexity,
- barrier count,
- register pressure,

- tensor core utilization,
- memory stall time.

Mixed Precision Matmul

Explore mixed precision matrix multiplication where inputs use lower precision and accumulation uses higher precision:

$$A, B \in \text{FP8} \quad \text{or} \quad A, B \in \text{BF16}, \quad C_{\text{acc}} \in \text{FP32}.$$

Study where conversion occurs:

$$\text{dtype conversion} \rightarrow \text{ALU work},$$

while matrix multiplication uses:

$$\text{tensor core work}.$$

The scheduling problem is to avoid starving tensor cores while conversion work runs.

Final Mental Model

Mosaic GPU and Pallas are designed around the idea that modern accelerator programming is no longer just about launching many threads and hoping occupancy hides latency. Hopper and Blackwell-style kernels need explicit orchestration of memory movement, shared memory layout, tensor core instructions, ALU work, warp groups, barriers, and sometimes block clusters.

The programmer should specify the meaningful performance structure:

tiles+access patterns+pipeline strategy+memory layout+hardware role assignment.

The DSL should generate the mechanical parts:

TMA descriptors+barrier plumbing+pipeline loops+copy scheduling+multicast instruction choreography.

For transformer workloads, this is especially powerful because the computations are structured and repeated. Attention, matmul, quantization, and fused kernels can often be expressed as compact tile programs with carefully controlled memory and compute schedules.

The final mental model is this: Pallas describes tiled computation from Python; Mosaic GPU lowers that description into hardware-aware programs that explicitly coordinate TMA, shared memory, WGMMMA, warp specialization, and synchronization so that modern GPU resources stay busy together rather than waiting on one another.